

Problem:1 Calculate $\sin(x)$ using the following series. Check the result for $x=30^\circ$.

$$\sin(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots$$

(1)

$$= \sum_{n=0}^{\infty} \frac{(-1)^n}{(2n+1)!} x^{2n+1}$$

Code:

Harshita Pulavarti

24CE10087

Python 3.13.3

```
import math

def taylor_sin(x_rad, tol=1e-10, max_terms=50):
    term = x_rad
    sinx = 0.0
    n = 0

    while n < max_terms and abs(term) > tol:
        sinx += term
        n += 1
        term *= -1 * x_rad**2 / ((2*n)*(2*n+1))

    return sinx

# x = 30 degrees
x_deg = 30
x_rad = math.radians(x_deg)

approx = taylor_sin(x_rad)

print("sin(30°) using Taylor series:", approx)
```

Output:

=
[sin\(30°\) using Taylor series: 0.5000000000202799](#)

Problem:2 Plot the following function for 5s using matplotlib.

$$f(t) = e^{-t} \cos(2\pi t) \quad (2)$$

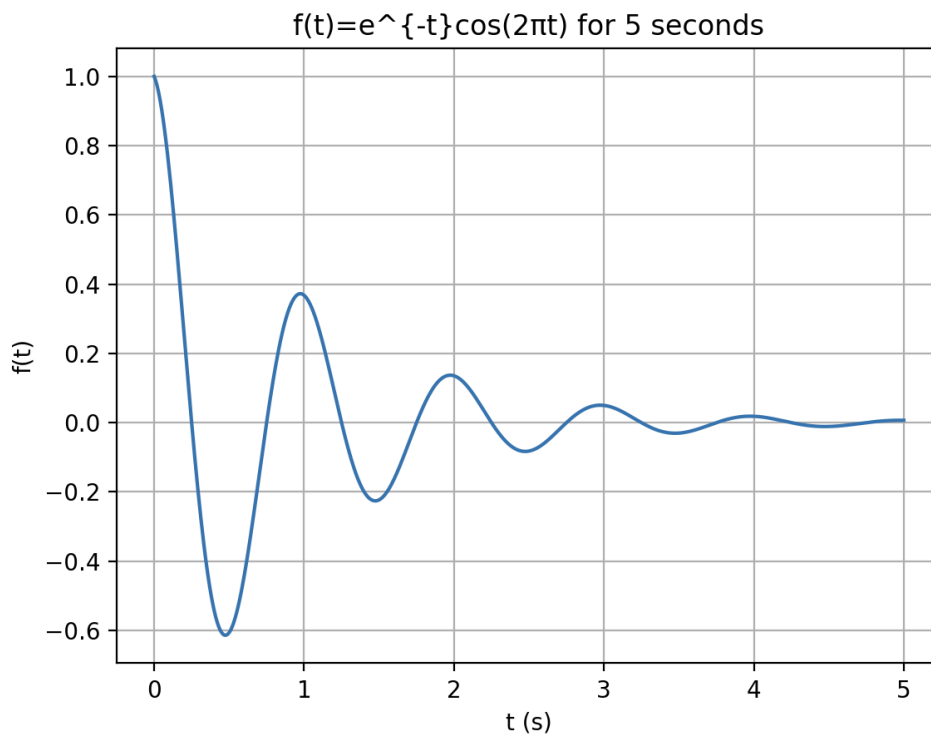
Code:

```
import numpy as np
import matplotlib.pyplot as plt

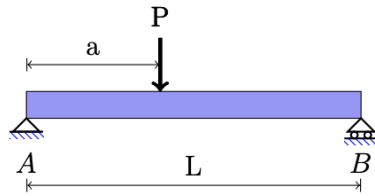
t = np.linspace(0,5,500)
f = np.exp(-t) * np.cos(2*np.pi*t)

plt.figure()
plt.plot(t,f)
plt.xlabel("t (s)")
plt.ylabel("f(t)")
plt.title("f(t)=e^{-t}cos(2πt) for 5 seconds")
plt.grid(True)
plt.show()
```

Output:



Problem:3 The simply supported beam with concentrated load is shown in Fig. 1. Calculate reactions, shear force, bending moment, slope and deflection. Also plot using matplotlib.



$$R_L = R_A = \frac{P(L-a)}{L} \quad R_R = R_B = \frac{Pa}{L}$$

$$M_{span} = \frac{Pab}{L} \quad M_{max} = \frac{PL}{4} \quad \text{at } a = \frac{L}{2}$$

$$\delta_P = \frac{Pa^2(L-a)^2}{3EI} \quad \delta_{max} = \frac{PL^3}{48EI} \quad \text{at } a = \frac{L}{2}$$

Figure 1: Simply supported beam with concentrated load

Code:

```
import numpy as np
import matplotlib.pyplot as plt

# Input
P = 10    # kN (example, change as needed)
L = 6     # m
a = 2     # m
E = 2e5   # kN/m^2 (convert if needed)
I = 8e-3  # m^4 (example)

b = L-a

# Reactions
RA = P*(L-a)/L
RB = P*a/L

print("RA =",RA)
print("RB =",RB)

# x array
x = np.linspace(0,L,400)
```

```

# Shear
V = np.piecewise(x, [x<a, x>=a], [lambda x: RA, lambda x: RA-P])

# Bending moment
M = np.piecewise(x,
    [x<a, x>=a],
    [lambda x: RA*x,
     lambda x: RA*x - P*(x-a)]
)
# Deflection and slope using formula (Appendix A)
deflection = np.zeros_like(x)
slope = np.zeros_like(x)

for i,xi in enumerate(x):
    if xi<=a:
        deflection[i] = -P*b*xi*(L**2 - b**2 - xi**2)/(6*L*E*I)
    else:
        deflection[i] = -P*b*xi*(L**2 - b**2 - xi**2)/(6*L*E*I) - P*(xi-a)**3/(6*E*I)

# Plot diagrams
plt.figure()
plt.plot(x,V)
plt.title("Shear Force Diagram")
plt.grid(True)
plt.show()

plt.figure()
plt.plot(x,M)
plt.title("Bending Moment Diagram")
plt.grid(True)
plt.show()

plt.figure()
plt.plot(x,deflection)
plt.title("Deflection Curve")
plt.grid(True)
plt.show()

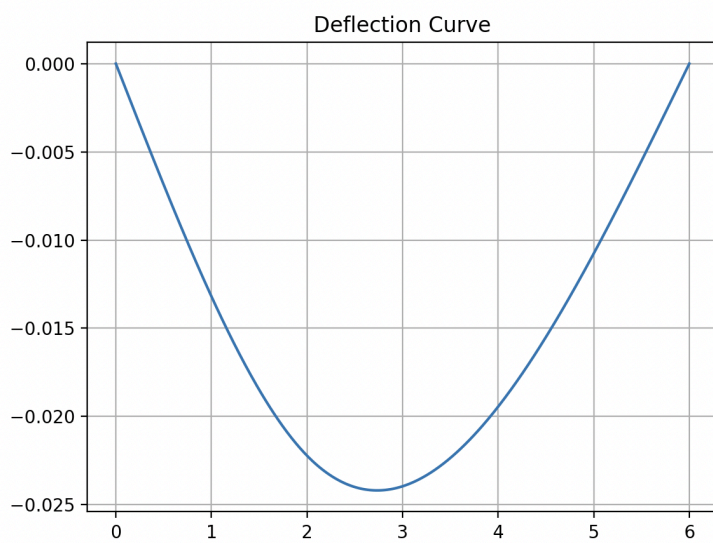
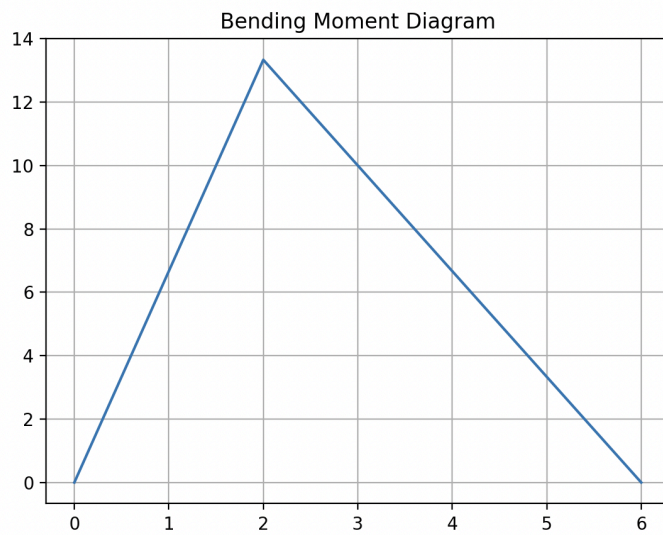
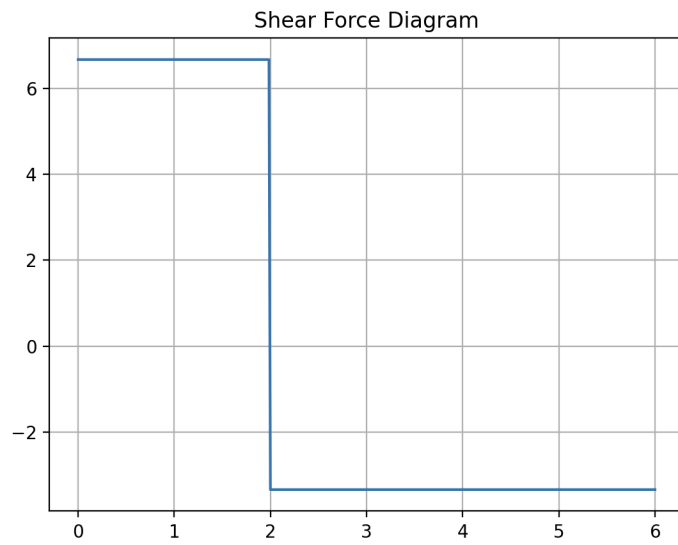
```

Output:

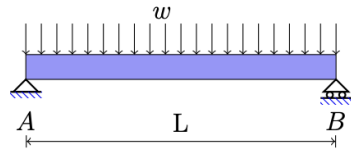
```

=
RA = 6.666666666666667
RB = 3.3333333333333335

```



Problem:4 The simply supported beam with UDL is shown in Fig. 2. Calculate reactions, shear force, bending moment, slope and deflection. Also plot using matplotlib.



$$R_L = R_A = \frac{wL}{2} \quad R_R = R_B = \frac{wL}{2}$$

$$M_{span} = \frac{wL^2}{8} = 0.125wL^2$$

$$\delta_{max} = \delta = \frac{5wL^4}{384EI} = 0.01302 \frac{wL^4}{EI}$$

Figure 2: Simply supported beam with UDL

Code:

```
import numpy as np
import matplotlib.pyplot as plt

# Inputs
w = 5    # kN/m
L = 6    # m
E = 2e5
I = 8e-3

# reactions
RA = RB = w*L/2

x = np.linspace(0,L,400)

# Shear
V = RA - w*x

# Bending moment
M = RA*x - w*x**2/2

# Deflection from Appendix-B
v = -w*x*(L**3 - 2*L*x**2 + x**3)/(24*E*I)
```

```
print("Maximum deflection =", np.min(v))
```

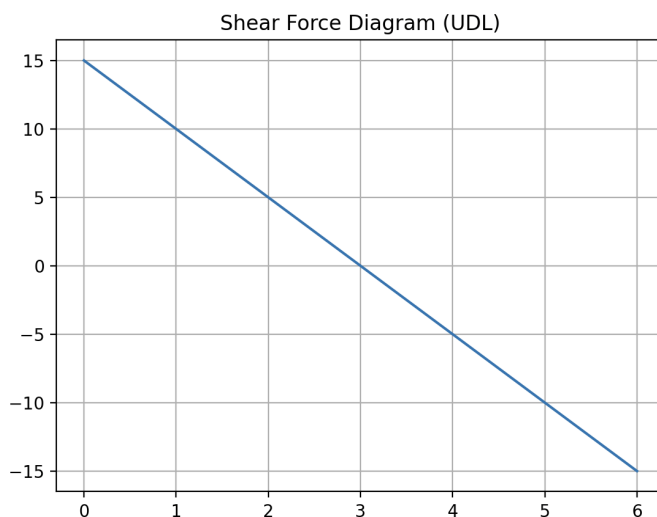
```
# Plotting  
plt.figure()  
plt.plot(x,V)  
plt.title("Shear Force Diagram (UDL)")  
plt.grid(True)  
plt.show()
```

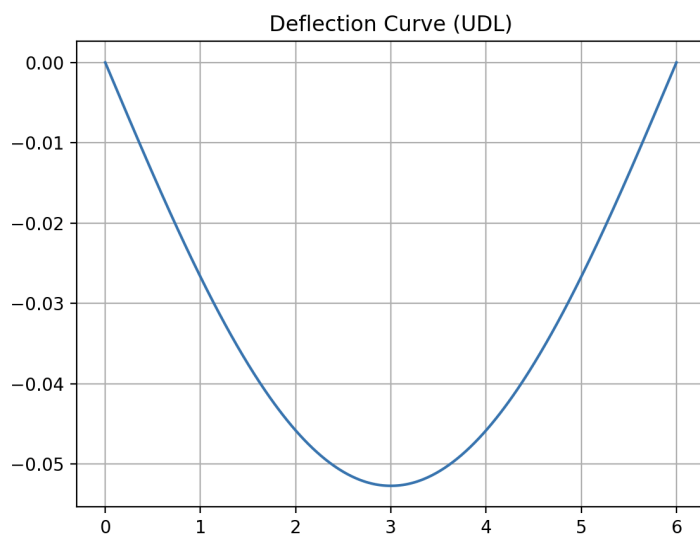
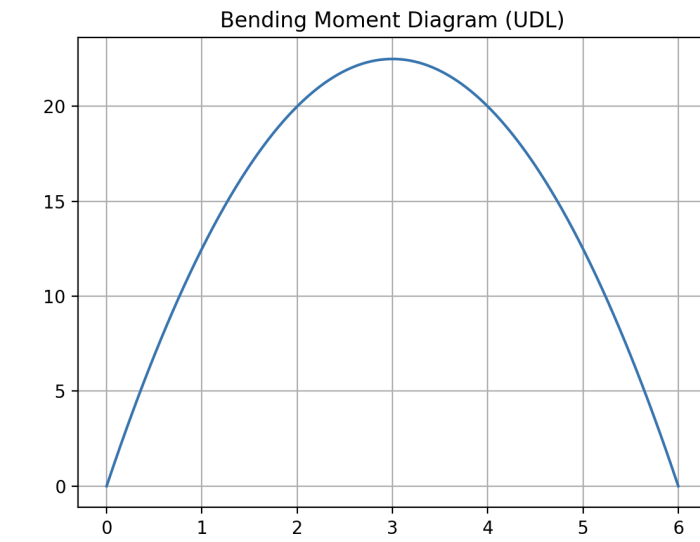
```
plt.figure()  
plt.plot(x,M)  
plt.title("Bending Moment Diagram (UDL)")  
plt.grid(True)  
plt.show()
```

```
plt.figure()  
plt.plot(x,v)  
plt.title("Deflection Curve (UDL)")  
plt.grid(True)  
plt.show()
```

Output:

Maximum deflection = -0.05273397750762399





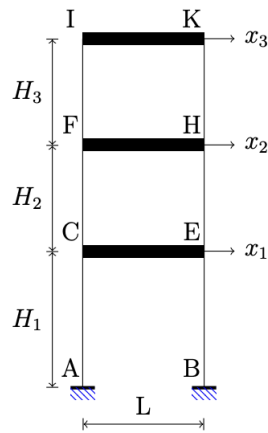


Figure 3: Three storeyed frame

Problem:5 The single bay three storeyed building frame is shown in Fig. 3. The length, $L = 4.0\text{m}$. The height (H_1) of first floor is 4.5m , the height (H_2) of second floor is 3.5m and the height (H_3) of roof is 3.5m . The size of each column is $300\text{mm} \times 400\text{mm}$ and modulus of elasticity, E is 20kN/mm^2 . The horizontal forces in each floor are $P_1 = 1000\text{kN}$, $P_2 = 1000\text{kN}$ and $P_3 = 1000\text{kN}$. Considering three degrees of freedom system,

- Compute stiffness matrix and load vector.
- Compute horizontal displacements in each floor.

Code:

```
import numpy as np
```

```
# Geometry
```

```
L = 4.0
```

```
H1 = 4.5
```

```
H2 = 3.5
```

```
H3 = 3.5
```

```
# Material
```

```
E = 20 # kN/mm2
```

```
E = E * 1e6 # convert to kN/m2
```

```
# Section
```

```
b = 0.3
```

```
D = 0.4
```

```
I = b*D**3/12
```

```
# Storey stiffness
```

```

def K(E,I,H):
    return 2*(12*E*I/H**3)

K1 = K(E,I,H1)
K2 = K(E,I,H2)
K3 = K(E,I,H3)

Kmat = np.array([
    [K1+K2, -K2, 0],
    [-K2, K2+K3, -K3],
    [0, -K3, K3]
])

print("Stiffness matrix:\n",Kmat)

# Load vector
P = np.array([1000e3,1000e3,1000e3]) # kN → N scale consistent

# Displacements
disp = np.linalg.solve(Kmat,P)

print("Floor displacements (m):\n",disp)

```

Output:

```

=
Stiffness matrix:
[[ 26340.51998224 -17912.53644315  0.          ]
 [-17912.53644315  35825.0728863  -17912.53644315]
 [  0.          -17912.53644315  17912.53644315]]
Floor displacements (m):
[355.95703125  467.61067708  523.4375      ]

```

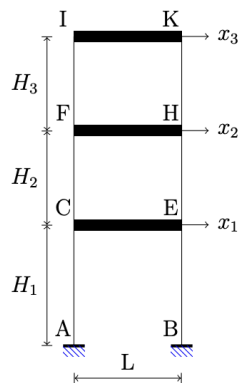


Figure 4: Three storeyed frame

Problem:6 The mass of each floor of a single bay three storeyed building frame as shown in Fig. 4 is 300.0 kg/m . The length, $L = 4.0\text{ m}$. The height (H_1) of first floor is 4.5 m , the height (H_2) of second floor is 3.5 m and the height (H_3) of roof is 3.5 m . The size of each column is $300\text{ mm} \times 400\text{ mm}$ and modulus of elasticity, E is 20 kN/mm^2 . Considering three degrees of freedom system,

- Compute mass and stiffness matrices.
- Compute natural frequencies and modes shapes

Code:

```
import numpy as np

# -----
# Geometry
# -----
L = 4.0      # m
H1 = 4.5     # m
H2 = 3.5     # m
H3 = 3.5     # m

# -----
# Material
# -----
E = 20       # kN/mm^2
# convert kN/mm2 → N/m2
E = E * 1e6 * 1e6 # 1 kN = 1000 N, 1 mm = 1e-3 m

# -----
# Section properties
# -----
b = 0.3      # m
D = 0.4      # m
I = b * D**3 / 12
```

```

# -----
# Story stiffness terms
# -----
def story_stiffness(E,I,H):
    return 2 * (12 * E * I / H**3)

K1 = story_stiffness(E,I,H1)
K2 = story_stiffness(E,I,H2)
K3 = story_stiffness(E,I,H3)

# -----
# Global stiffness matrix
# -----
K = np.array([
    [K1 + K2, -K2, 0],
    [-K2, K2 + K3, -K3],
    [0, -K3, K3]
])

print("Stiffness matrix K =\n",K)

# -----
# Mass matrix
# -----
w = 300.0    # kg/m
m1 = w * L
m2 = w * L
m3 = w * L

M = np.diag([m1, m2, m3])

print("\nMass matrix M =\n",M)

# -----
# Eigenvalue problem
# -----
# Solve  $[K - \omega^2 M]\{\phi\}=0$ 
evals, evecs = np.linalg.eig(np.linalg.inv(M).dot(K))

# natural circular frequencies
omega = np.sqrt(evals)

# natural frequencies in Hz
freq = omega / (2 * np.pi)

print("\nNatural circular frequencies  $\omega$  (rad/s):\n",omega)
print("\nNatural frequencies f (Hz):\n",freq)

```

```
# mode shape normalization (first component = 1)
modes = evecs / evecs[0,:]

print("\nMode shapes (normalized):\n",modes)
```

Output:

```
=
Stiffness matrix K =
[[ 2.63405200e+10 -1.79125364e+10  0.00000000e+00]
 [-1.79125364e+10  3.58250729e+10 -1.79125364e+10]
 [ 0.00000000e+00 -1.79125364e+10  1.79125364e+10]]

Mass matrix M =
[[1200.    0.    0.]
 [  0. 1200.    0.]
 [  0.    0. 1200.]]

Natural circular frequencies  $\omega$  (rad/s):
[6795.58304158 4329.42250625 1344.59150439]

Natural frequencies f (Hz):
[1081.55063226 689.0489926 213.99838436]

Mode shapes (normalized):
[[ 1.         1.         1.        ]
 [-1.62318825  0.21481273  1.34939061]
 [ 0.77527416 -0.84011377  1.53534715]]
```